Professional Issues: Project Management

The professional issues that had the most tangible effect on the project were management, including estimating time and resources at the start, revising those estimates when situations changed and consulting the supervisor to keep work on course. Reflecting on the development over this academic year, it is clear that any decisions made around planning, prioritisation and communication moulded the outcome as much as the technical choices.

Adapting to the Changed Technology Stack

The project requirements were originally for Apache Hadoop and the MapReduce paradigm. In the first supervisor meeting, the technology was changed to Apache Spark with the reasoning being that it was a more modern and better-suited alternative to the range of problems being tackled. While those 4 core problems from the project specification remained the same, the shift meant that every implementation detail had to be adjusted and reevaluated.

The PySpark documentation itself was well written, and the syntax was not difficult, as it was a Python library, and I was well-versed with Python before this project. The more difficult aspect was understanding the internal mechanisms of how things like the lazy evaluation defer computation until an action is triggered, how the DAG scheduler pipelines transformations and why a shuffle stage can dominate the runtime of otherwise efficient workloads. These concepts took time to internalise, and it was only after weeks of reading and local iterative experimentation that confidence grew enough to begin working on the cluster.

Infrastructure Reliability and Lost Work

A significant management challenge arose from the university's Big Data cluster, which experienced repeated hardware issues in my experience throughout the course of the project. The server went down two to three times during the first semester, and an extended maintenance took place, preventing usage for a prolonged period of time. On many occasions, long-running Spark workloads that were executing using a lower resource configuration and were therefore taking quite a long time to complete, had been completely lost when the server went down or crashed. Even jobs that ran inside tmux sessions did not help, as the sessions were terminated when the server would go down.

These disruptions taught a practical lesson about contingency planning. My timeline originally did not include any buffer for server downtime, and in hindsight, this was a clear oversight due to my lack of experience using external servers that would either crash or need maintenance-related downtime. In a professional environment, infrastructure outages are to be expected, not exceptional, and the project plans must therefore account for them. I adapted by restructuring my testing approach: starting with lower configurations and scaling up so that the most demanding runs were attempted only after the shorter ones had been completed. I also learned to communicate proactively with the CIM help desk, raising tickets for every issue encountered. Their responses provided invaluable support, with detailed information about the cluster architecture, spark-submit configuration parameters and the resource limits.

Responsible Use of Share Cluster Environment

The cluster consisted of ten virtual machine executors hosted on a single physical server, shared among various users throughout the university. The CIM help desk informed me that the theoretical maximum was 10 executors, with each having 4 cores and 24 GB of memory. This was a huge amount of resources, but hygienic cluster usage would always need to be taken into consideration.

During performance testing stages, I only used the maximum cluster capacity, in terms of executors and cores, as memory increases did not seem to affect the runtime, only during late hours when I was confident no other users were running intense jobs. The memory scaling experiments reinforced this approach, showing that allocating more than 4 to 6 GB of memory per executor provided negligible improvement. Rather than using the maximum available memory for more runs, I chose to only use what was optimal for both the run and the server. This consideration reflects the responsibility that computing practitioners must consider when operating on shared infrastructure, whether that is a university cluster or a production cloud environment; the same concepts and concerns would be taken into account.

Scope Management and Trade-offs

The project scope was defined by the project requirements, which specified 4 unique problems of increasing complexity. However, several mid-project decisions required pragmatic trade-offs in scope. One such example was the decision to remove precipitation from the K-Means feature set. After spending several days attempting to incorporate added weather features, it became apparent that precipitation was skewing the clustering output when visualised via the global map. Persisting with the original feature set for the sake of completeness, rather than assuming, was one of the more valuable lessons learned during this project and taught me more about how the K-Means algorithm works.

Another trade-off involved switching the output file format from Parquet to CSV. Parquet is a more efficient format that was recommended to me to use for the distributed storage, but due to the output needing to be transferred to my local machine for visualisation, and the SSH terminal did not support displaying my plots directly, a CSV export proved far more practical. The output file was small enough that the efficiency difference was negligible, and being able to open and inspect results immediately outweighed the theoretical storage advantage that could have been gained.

A final consideration was the potential use of AWS Academy for cloud resources. Despite requesting access several times across the supervisor meetings, delays meant that by the time it would have been available, the remaining timeline made it impractical to incorporate it into the project. This mirrors the professional reality that not every planned resource will materialise on schedule, and effective management requires the flexibility and constant adjustment to deal with situations accordingly.

Reflection

Upon reflection, across the several meetings with my supervisor and the entire development cycle, the most invaluable management lesson from this project is that planning is not an activity that is done at the start, but in reality takes many different twists and turns during the course of a project. Effective management requires continuous reassessment of time estimates when dealing with new situations. The use of GitLab issues with

clearly listed required actions and expected deliverables, along with feature branches to segment the work, provided a structured approach to managing the expanding workload through the project's lifecycle.

If I were advising a future student undertaking a similar project using the university cluster, I would recommend a few things. Firstly, set up tmux sessions when running any performance testing or code test runs, as you can be productive when you detach from the session and let it run in the background. Secondly, plan for the cluster to be unavailable at the worst possible times to allow for breathing room when running the performance testing portion of the project. Finally, communicate with the CIM help desk as early and often as necessary, as their support was very useful when running into problems or questions regarding my project or the cluster itself.